DLottery Product Requirements Document

Oct 30, 2026 · @Nicole

This document specifies the minimum viable product (MVP) for DLottery, a decentralized lottery on an EVM-compatible blockchain, covering the smart contract, backend and frontend.

Document control

Field	Value
Product	DLottery (Decentralized Lottery)
Document type	Product Requirements Document (PRD)
Scope	Minimum Viable Product (MVP)
Version	1.1

Conventions. The key words "must", "must not", "should" and "may" are used as defined in RFC 2119. Requirement identifiers (for example SC-01, BE-01, FE-01) are provided for traceability.

1. Product Overview

1.1 Product Description

DLottery is a decentralized lottery application deployed on an EVM-compatible blockchain. Users purchase numbered tickets with the USD8 token; each draw settles on-chain, and all funds are held and released by the smart contract.

1.2 System Components

Component	Responsibility
Smart Contract	Manages lottery rounds, ticket purchases, random draw settlement, prizes, refunds, rollover funds and DAO fees. It is the authoritative source of lottery and financial state.

Component	Responsibility
Frontend	Provides wallet connection, ticket purchase, draw interaction, results, refunds and draw history.
Backend	Acts as a blockchain event indexer and history/query service.
USD8	ERC-20 token used to purchase lottery tickets.

1.3 Core Business Rules

Rule	Requirement
Payment token	USD8
Ticket price	10 USD8
Maximum participants	5 per draw
Ticket numbers	1–10
Tickets per wallet	1 per draw
Draw duration	24 hours
Minimum participants	Configurable (for example, 2)
Draw trigger	5 participants reached, or 24-hour timeout
Insufficient participants	Draw is cancelled and refunds are enabled
Winner	The participant whose ticket number matches the lucky number
No winner	Prize pool rolls over to the next draw
DAO fee	5% of the winner's net profit

2. Smart Contract Requirements

The smart contract is the authoritative source of all lottery and financial state.

2.1 Draw Management

ID	Requirement
SC-01	The contract must initialize the lottery configuration.
SC-02	The contract must create a unique Draw ID for each round.
SC-03	The contract must start a new draw and record its start time.
SC-04	The contract must ensure the previous draw is finalized before a new draw starts.
SC-05	The contract must carry forward rollover funds to the next draw.
SC-06	The contract must manage draw state transitions.

2.2 Ticket Management

ID	Requirement
SC-07	The contract must accept USD8 as ticket payment.
SC-08	The contract must validate the draw status and the 24-hour deadline before accepting a purchase.
SC-09	The contract must enforce the maximum participant limit.
SC-10	The contract must allow only one ticket per wallet per draw.
SC-11	The contract must assign a unique ticket number from 1 to 10.
SC-12	The contract must prevent duplicate ticket numbers within a draw.
SC-13	The contract must emit an event for each ticket purchase.

2.3 Lottery Settlement

ID	Requirement
SC-14	The contract must allow the draw only after the round is eligible for settlement.
SC-15	The contract must verify the minimum participant requirement.
SC-16	The contract must generate the lucky number.
SC-17	The contract must determine whether any participant's ticket matches the lucky number.
SC-18	The contract must record the winner when a match exists.
SC-19	The contract must mark the round as `NO_WINNER` when no match exists.
SC-20	The contract must roll over the prize pool when there is no winner.

2.4 Prize and Refund Management

ID	Requirement
SC-21	Only the winner may claim the prize.
SC-22	The contract must calculate the winner's net profit.
SC-23	The contract must transfer the DAO fee to the DAO.
SC-24	The contract must transfer the remaining prize to the winner.
SC-25	Refunds must be enabled only for cancelled rounds.
SC-26	Each participant may claim their ticket principal only once.

2.5 Required Events

Event	Emitted when
`DrawStarted`	A new draw begins
`TicketBought`	A participant purchases a ticket
`LotteryDrawn`	The lucky number is generated and the result recorded
`DrawCancelled`	A draw is cancelled for insufficient participants
`PrizeClaimed`	The winner claims the prize
`RefundClaimed`	A participant claims a refund

3. Backend Requirements

The backend is a passive blockchain indexer and query service. It must not determine lottery results or control funds.

3.1 Deployment Architecture

The backend must be deployed as a containerized service and implemented in **one** of the following languages:

- Golang
- Rust

The choice between them is an engineering decision. Either implementation must provide the same functional capabilities and APIs.

ID	Deployment requirement
BE-01	Containerized deployment
BE-02	Blockchain RPC connectivity
BE-03	MySQL or PostgreSQL connectivity
BE-04	REST API service
BE-05	Blockchain event listener
BE-06	Production-ready logging and error handling

3.2 Blockchain Event Listener

The listener must continuously monitor these lottery contract events: draw started, ticket purchased, lottery drawn, draw cancelled, prize claimed and refund claimed.

ID	Listener requirement
BE-07	RPC reconnection
BE-08	Event processing
BE-09	Block confirmation handling
BE-10	Chain reorganization protection
BE-11	Retry and error handling

3.3 Database Synchronization

The backend must store and maintain lottery history. Records must be updated idempotently so that no duplicate data is created.

Field	Description
Draw ID	Unique identifier of the draw
Draw status	Current state of the draw
Start / end time	Draw opening and closing timestamps
Ticket price	Price per ticket in USD8
Participant count	Number of tickets sold
Lucky number	Number generated at settlement
Winner address	Wallet address of the winner, if any
Total prize pool	Pool including rollover
DAO fee	Fee paid to the DAO
Winner prize	Amount claimable by the winner

3.4 API Specification

Endpoint	Method	Returns
`/api/v1/draws/current`	GET	Current draw information, draw status, remaining time, participant information, ticket allocation and prize pool
`/api/v1/draws/history`	GET	Paginated historical draw records: Draw ID, status, start/end time, lucky number, winner, total pool, DAO fee and winner prize

4. Frontend Requirements

4.1 Wallet

ID	Requirement
FE-01	Connect a Web3 wallet.
FE-02	Display the connected wallet address.
FE-03	Display the user's USD8 balance.
FE-04	Detect the user's participation status in the current draw.
FE-05	Enable or disable actions according to draw status.

4.2 Active Draw Dashboard

The dashboard must display the current Draw ID, draw status, ticket price, participant count, remaining time, total prize pool and ticket number allocation. The allocation table must list numbers 1–10 with the wallet address holding each number.

4.3 Ticket Purchase

The frontend must perform the following steps in order:

1. Check the user's USD8 balance.
2. Check the USD8 allowance.
3. Request approval when the allowance is insufficient.
4. Submit the ticket purchase transaction.

5. Display the transaction status.
6. Display the assigned ticket number.
7. Prevent another purchase from the same wallet during the round.

4.4 Lottery Draw

When the draw is eligible, the frontend must display the **Perform Lottery Draw** action, show a short loading animation, wait for blockchain confirmation and then display the final result.

4.5 Result and Refund Views

View	Must display
Winner result	Winner wallet, lucky number, total prize pool, DAO fee, winner claimable amount and the prize claim action
No-winner result	Lucky number, no-winner status, rollover amount and next draw information when available
Refund (cancelled draw)	Cancellation status, **Claim Refund** action for eligible participants, refund transaction status; the action is disabled after a successful claim

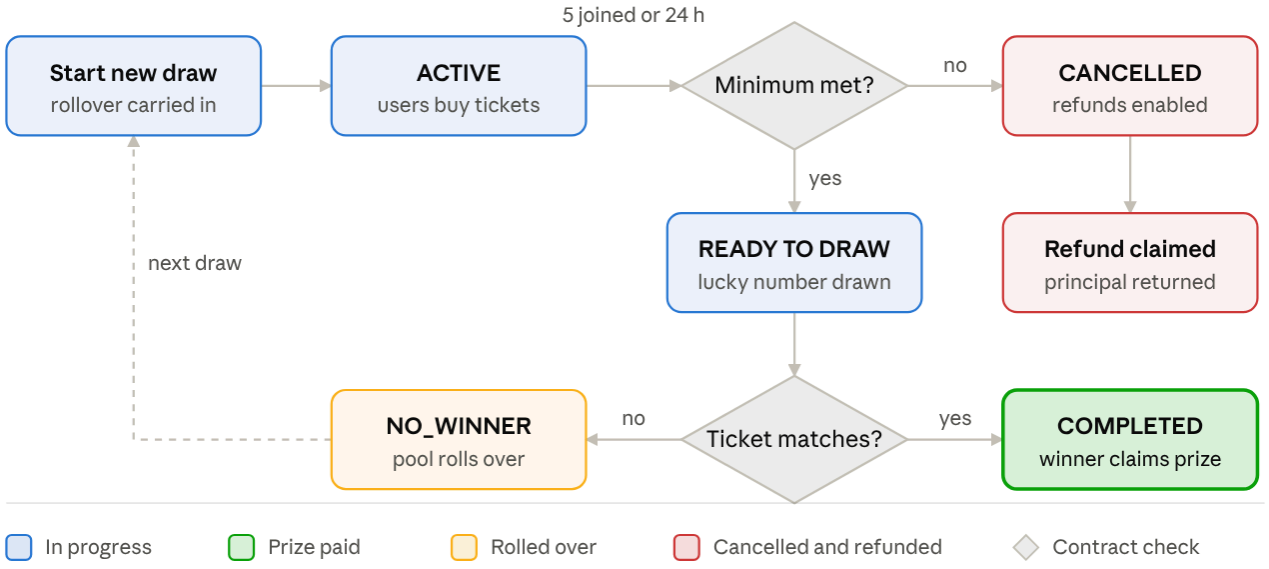
4.6 Draw History

The frontend must display historical lottery results, including Draw ID, status, time, lucky number, winner, total pool, DAO fee and winner prize.

5. Core Flows and Calculations

5.1 Draw Lifecycle

A draw closes when 5 participants join or 24 hours elapse; it is then settled if the minimum is met, otherwise cancelled with refunds.



Draw lifecycle · 7 states, 2 decisions

A no-winner draw rolls its pool into the next draw; a cancelled draw returns each participant's principal.

5.2 Ticket Purchase Flow

1. User connects wallet and views the active draw.
2. Frontend checks the USD8 balance.
3. Frontend checks the USD8 allowance; if insufficient, the user approves USD8.
4. User submits the ticket purchase.
5. Contract transfers USD8 and assigns a unique ticket number.
6. Frontend confirms the purchase and updates the display.

5.3 Refund Flow

1. 24 hours elapse with fewer participants than the minimum.
2. The draw is marked as cancelled.
3. The participant connects their wallet and selects **Claim Refund**.
4. The contract verifies the participant.
5. The contract returns the ticket principal.
6. The refund is marked as claimed.

5.4 Prize Pool and Winner Profit

Where *T* = ticket price, *N* = number of participants, *R* = rollover amount and *P_total* = total prize pool:

$$P_{total} = R + (N \times T)$$

The winner's original ticket principal is excluded from the DAO fee:

$$\text{Net Profit} = P_{total} - T$$

5.5 DAO Fee and Winner Payout

The DAO fee is 5% of the winner's net profit:

$$\text{DAO Fee} = \max(0,\ \text{Net Profit} \times 5\%)$$

$$\text{Winner Claimable} = P_{total} - \text{DAO Fee} = T + (\text{Net Profit} \times 95\%)$$

5.6 Worked Example

With a rollover of 20 USD8, 5 participants and a ticket price of 10 USD8, the winner receives 67 USD8 and the DAO receives 3 USD8.

Item	Calculation	Amount (USD8)
Total prize pool	20 + (5 × 10)	70
Winner net profit	70 − 10	60
DAO fee	60 × 5%	3
Winner claimable	70 − 3	67